

트랜스포머(Transformer)

[등장 배경](#)

[트랜스포머?](#)

[특징](#)

[포지셔널 인코딩\(Positional Encoding\)](#)

[셀프 어텐션\(Self Attention\)](#)

[주요 하이퍼파라미터](#)

[기존 모델과의 차이점](#)

[기존 모델 특징](#)

[차이점](#)

[순차적 처리 vs 병렬 처리](#)

[장거리 의존성 문제](#)

[계산 복잡도](#)

[메모리 사용](#)

[트랜스포머의 장점](#)

[병렬 처리 능력](#)

[장거리 의존성 문제 해결](#)

[모델 구조의 유연성](#)

[전이 학습\(Transfer Learning\)의 용이성](#)

[해석 가능성](#)

[트랜스포머 등장 이후의 변화](#)

[대규모 언어 모델의 등장](#)

[전이 학습의 보편화](#)

[멀티모달 AI의 발전](#)

[AI의 창의성 향상](#)

[결론](#)

[참고자료](#)

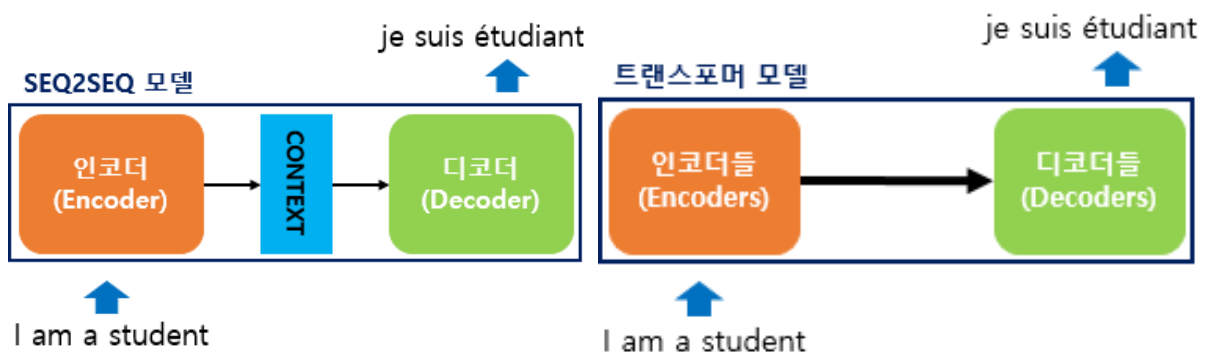
등장 배경

- 기존의 seq2seq 모델은 인코더-디코더 구조로 구성
 - 인코더는 입력 시퀀스를 하나의 벡터 표현으로 압축
 - 디코더는 이 벡터 표현을 통해서 출력 시퀀스를 만들
- 인코더-디코더 구조는 인코더가 입력 시퀀스를 하나의 벡터로 압축하는 과정에서 입력 시퀀스의 정보가 일부 손실된다는 단점이 존재했고 이를 보정하기 위해 어텐션이 사용 됨

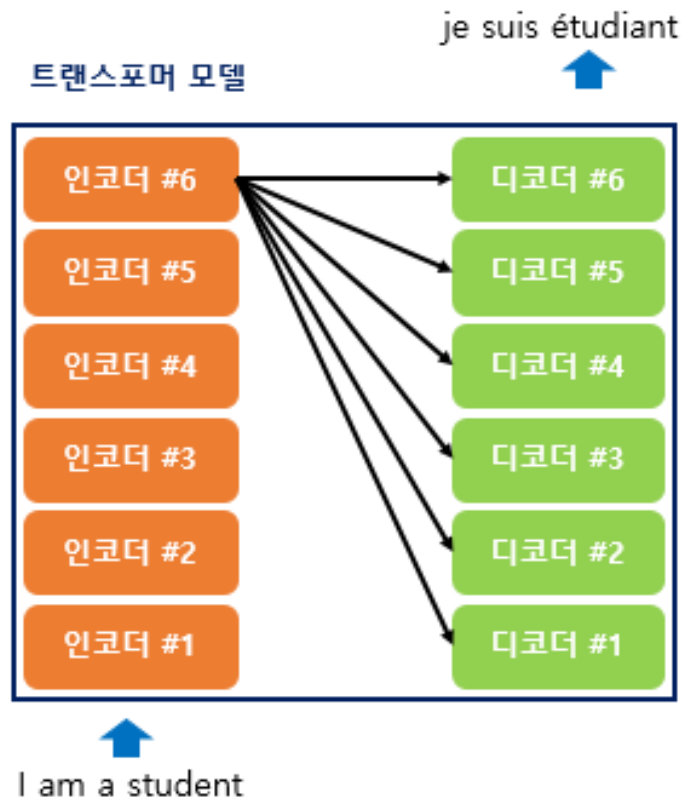
- 어텐션을 RNN의 보정을 위한 용도로서 사용하는 것이 아니라 어텐션만으로 인코더와 디코더를 만들어보면 어떨까?

트랜스포머?

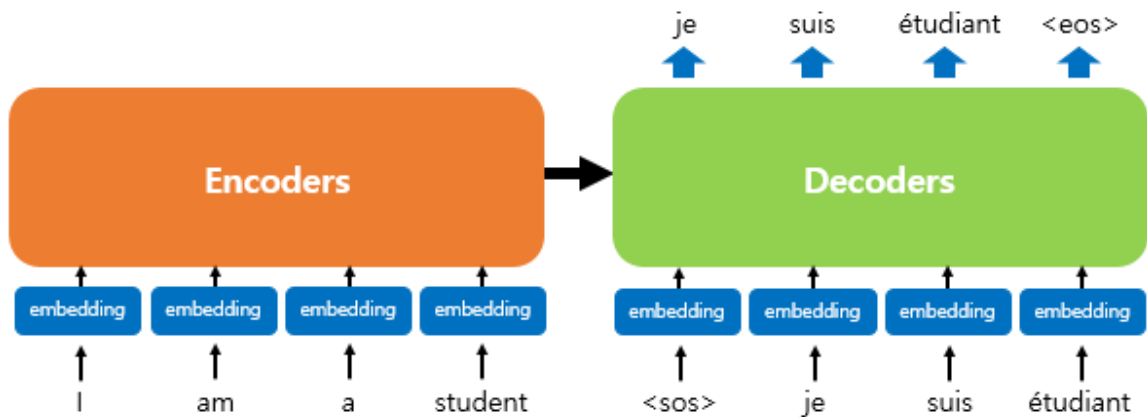
- 2017년 구글이 발표한 논문인 "Attention is all you need"에서 나온 모델
- 기존의 seq2seq의 구조인 인코더-디코더를 따르면서도, 논문의 이름처럼 어텐션 (Attention)만으로 구현한 모델
- 이 모델은 RNN을 사용하지 않고, 인코더-디코더 구조를 설계하였음에도 번역 성능에서도 RNN보다 우수한 성능을 보여줌



- 이전 seq2seq 구조에서는 인코더와 디코더에서 각각 하나의 RNN이 t개의 시점(time step)을 가지는 구조였다면 이번에는 인코더와 디코더라는 단위가 N개로 구성되는 구조
- 트랜스포머를 제안한 논문에서는 인코더와 디코더의 개수를 각각 6개 사용함



- 인코더로부터 정보를 전달받아 디코더가 출력 결과를 만들어내는 구조

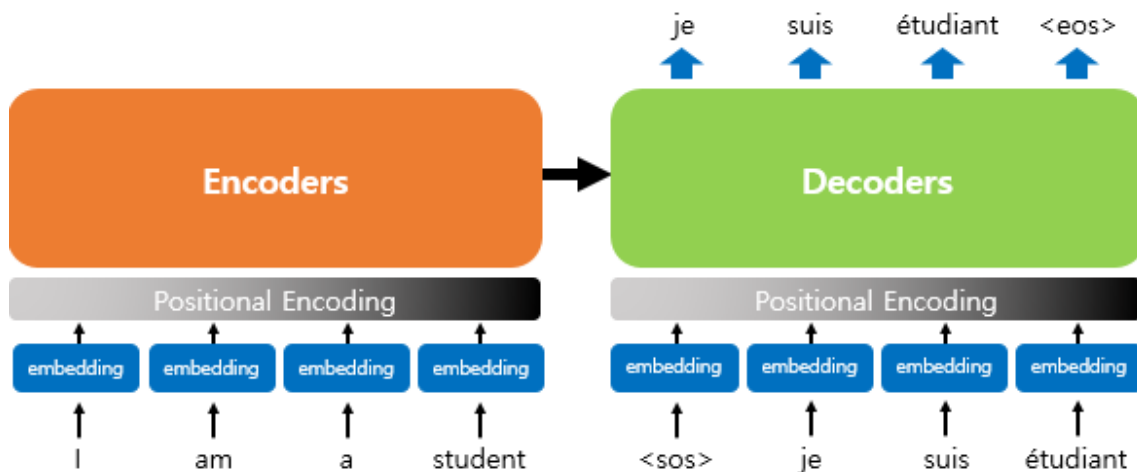


- 디코더는 마치 기존의 seq2seq 구조처럼 시작 심볼 <sos>를 입력으로 받아 종료 심볼 <eos>가 나올 때까지 연산을 진행함
- RNN은 사용되지 않지만 여전히 인코더-디코더의 구조는 유지되고 있음
- 트랜스포머의 인코더와 디코더는 단순히 각 단어의 임베딩 벡터들을 입력받는 것이 아니라 임베딩 벡터에서 조정된 값을 입력받음

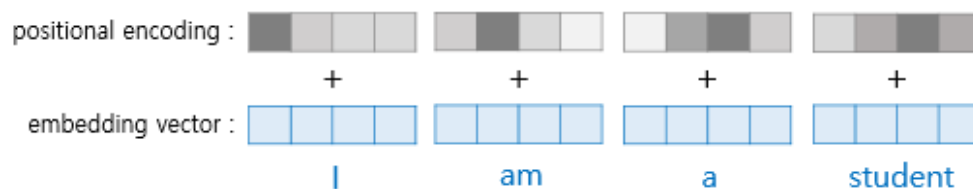
특징

포지셔널 인코딩(Positional Encoding)

- RNN이 자연어 처리에서 유용했던 이유는 단어의 위치에 따라 단어를 순차적으로 입력 받아서 처리하는 RNN의 특성으로 인해 각 단어의 위치 정보(position information)를 가질 수 있다는 점
- 트랜스포머는 단어 입력을 순차적으로 받는 방식이 아니므로 단어의 위치 정보를 얻기 위해서 각 단어의 임베딩 벡터에 위치 정보들을 더하여 모델의 입력으로 사용하는데, 이를 포지셔널 인코딩(positional encoding)이라고 함



- 임베딩 벡터가 인코더의 입력으로 사용되기 전 포지셔널 인코딩값이 더해지는 과정은 아래와 같음



- 트랜스포머는 위치 정보를 가진 값을 만들기 위해서 아래의 두 개의 함수를 사용함

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

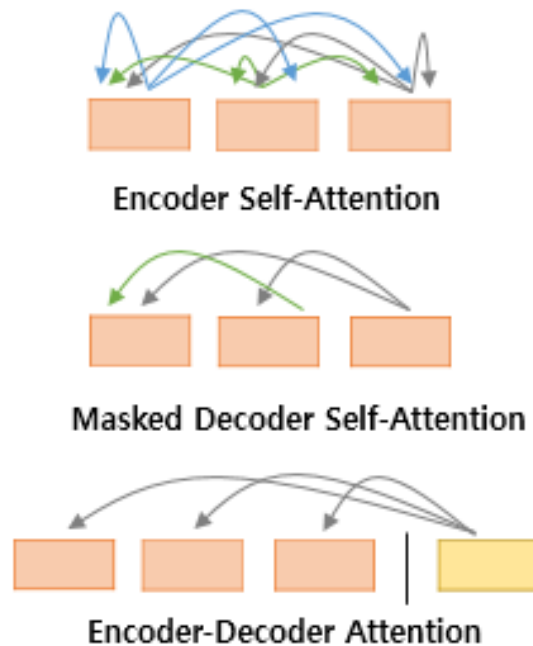
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

- 포지셔널 인코딩 방법을 사용하면 순서 정보가 보존되는데, 예를 들어 각 임베딩 벡터에 포지셔널 인코딩의 값을 더하면 같은 단어라고 하더라도 문장 내의 위치에 따라서 트랜스포머의 입력으로 들어가는 임베딩 벡터의 값이 달라짐

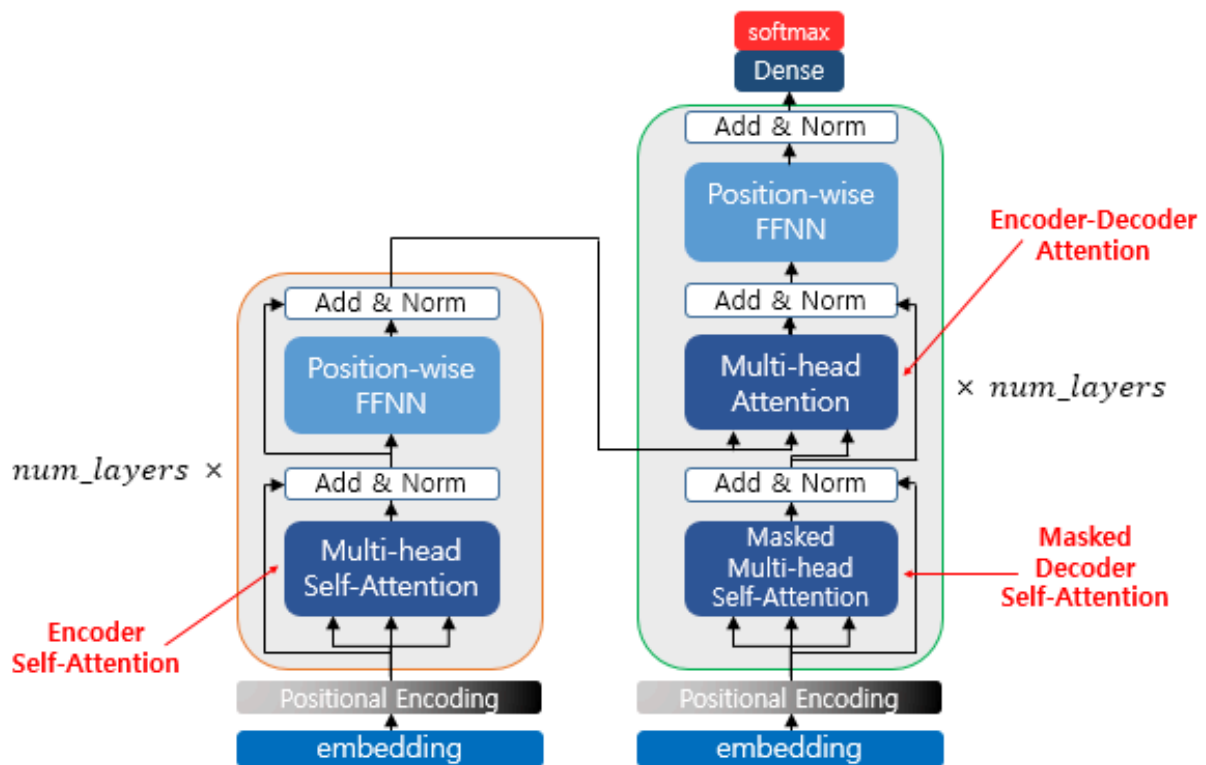
- 포지셔널 인코딩을 통해 트랜스포머의 입력은 순서 정보가 고려된 임베딩 벡터가 됨

셀프 어텐션(Self Attention)

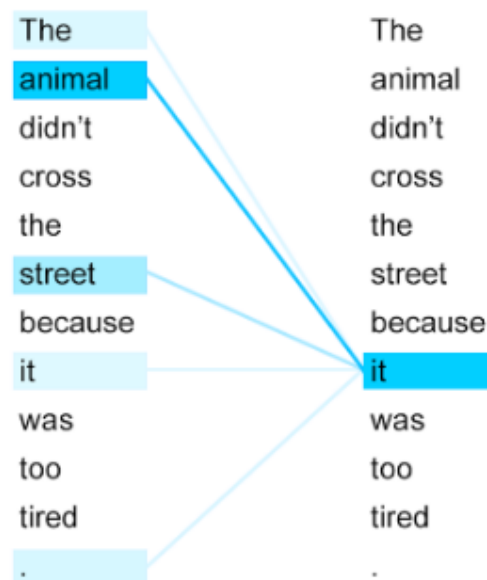
- 단어의 관계를 배우는 방법으로 병렬 처리가 가능해졌고, 장거리 의존성(long-range dependency) 문제를 해결함



- 첫번째 그림인 셀프 어텐션은 인코더에서 이루어지지만, 두번째 그림인 셀프 어텐션과 세번째 그림인 인코더-디코더 어텐션은 디코더에서 이루어짐
- 셀프 어텐션은 본질적으로 Query, Key, Value가 동일한 경우를 말함
- 세번째 그림 인코더-디코더 어텐션에서는 Query가 디코더의 벡터인 반면에 Key와 Value가 인코더의 벡터이므로 셀프 어텐션이라고 부르지 않음



- 셀프 어텐션은 입력 문장 내의 단어들끼리 유사도를 구하므로써 그것(it)이 동물 (animal)과 연관되었을 확률이 높다는 것을 찾아냄



예를 들어 "은행"이라는 단어를 셀프 어텐션 메커니즘으로 처리한다고 가정할 때, "은행" 주변에 "돈", "계좌" 같은 단어가 있다면, 최종 출력은 금융기관으로서 은행의 의미를 더 강하게 반영할 것임. 반면 "나무", "공원" 같은 단어가 주변에 있다면, 최종 출력은 나무로서 은행의 의미를 더 강하게 반영할 것임

- 셀프 어텐션의 장점은 다음과 같음
 - (1) 문장 전체 고려: 문장 내 모든 단어 쌍 사이의 관계를 직접 계산
 - (2) 병렬 처리: 모든 단어에 대해 동시에 계산이 가능
 - (3) 해석 가능성: 어텐션 가중치를 시각화하여 모델의 동작 과정을 이해할 수 있음

예를 들어, "나는 맛있는 사과를 먹었다"라는 문장에서 '맛있는'이라는 단어를 이해할 때, 트랜스포머는 이 단어가 '사과'와 가장 관련이 깊다고 판단함. 이렇게 각 단어마다 다른 단어들과의 관계를 계산하면, 전체 문장의 의미를 정확하게 이해할 수 있음

주요 하이퍼파라미터

| $d_{model} = 512$

- 트랜스포머의 인코더와 디코더에서의 정해진 입력과 출력의 크기를 의미
- 임베딩 벡터의 차원 또한 d_{model} 이며, 각 인코더와 디코더가 다음 층의 인코더와 디코더로 값을 보낼 때에도 이 차원을 유지함
- 논문에서는 512

| num_layers = 6

- 트랜스포머에서 하나의 인코더와 디코더를 층으로 생각하였을 때, 트랜스포머 모델에서 인코더와 디코더가 총 몇 층으로 구성되었는지를 의미
- 논문에서는 인코더와 디코더를 각각 총 6개 쌓음

| num_heads = 8

- 트랜스포머에서는 어텐션을 사용할 때, 여러 개로 분할해서 병렬로 어텐션을 수행하고 결과값을 다시 하나로 합치는데 이때 병렬의 개수를 의미함

| $d_{ff} = 2048$

- 트랜스포머 내부에는 피드 포워드 신경망이 존재하며 해당 신경망의 은닉층의 크기를 의미
- 피드 포워드 신경망의 입력층과 출력층의 크기는 d_{model}

기존 모델과의 차이점

기존 모델 특징

- RNN
 - 순차적 데이터를 처리하기 위해 설계된 신경망
 - 이전 단계의 정보를 현재 단계로 전달해 계산에 활용
 - 하지만 먼 과거의 정보를 효과적으로 활용하지 못하는 한계
- LSTM
 - RNN의 장거리 의존성 한계를 해결하기 위해 개발된 모델
 - 정보를 선택적으로 기억하고 잊을 수 있는 구조로 긴 시계열 데이터의 예측 정확도 개선
- GRU
 - LSTM을 단순화한 모델로, LSTM과 비슷한 성능을 보이면서 계산 효율성 개선

차이점

순차적 처리 vs 병렬 처리

- 기존 모델 : RNN, LSTM, GRU는 순차적으로 데이터를 처리. 마치 책을 한 단어씩 차례대로 읽는 것과 같음
- 트랜스포머 : 문장의 모든 단어를 동시에 처리. 책의 전체 페이지를 한 번에 훑어보는 것과 비슷

장거리 의존성 문제

- 기존 모델 : 문장이 길어질수록 먼 거리에 있는 단어 간의 관계를 포착하기 어려움. LSTM과 GRU는 이 문제를 어느 정도 개선했지만, 여전히 한계가 있음
- 트랜스포머 : 셀프 어텐션 메커니즘을 통해 문장 내 모든 단어 간의 관계를 직접적으로 계산하므로, 장거리 의존성 문제를 효과적으로 해결함

계산 복잡도

- 기존 모델 : 순차적 처리로 인해 문장 길이에 비례하여 계산 복잡도가 증가. 즉, 문장이 길어질수록 처리 시간이 크게 늘어남
- 트랜스포머 : 병렬 처리가 가능해 문장 길이에 덜 민감함. 다만, 셀프 어텐션 연산의 복잡도는 문장 길이의 제곱에 비례

메모리 사용

- 기존 모델 : RNN 계열 모델은 이전 상태를 기억하기 위한 내부 메모리를 사용함. 특히 LSTM은 장기 기억을 위한 복잡한 게이트 구조를 가짐
- 트랜스포머 : 별도의 메모리 구조 없이 어텐션 메커니즘만으로 정보를 처리

트랜스포머의 장점

병렬 처리 능력

- 기존 모델과 달리, 문장의 모든 단어를 동시에 처리할 수 있어 학습 속도가 매우 빠름
- 이는 대규모 데이터셋에서의 학습을 가능케 하여, 모델의 성능을 개선함

장거리 의존성 문제 해결

- 셀프 어텐션 메커니즘을 통해 문장 내 멀리 떨어진 단어 사이의 관계도 직접적으로 모델링
- 이는 장문의 이해나 복잡한 문맥 파악에 큰 도움이 됨

모델 구조의 유연성

- Transformer는 인코더와 디코더라는 두 가지 주요 구성 요소로 이루어져 있습니다. 인코더는 입력 데이터(예: 원본 문장)를 받아 이를 컴퓨터가 이해할 수 있는 형태로 변환함. 마치 통역사가 외국어를 듣고 그 의미를 이해하는 과정과 비슷합니다. 반대로 디코더는 인코더가 변환한 정보를 받아 원하는 출력(예: 번역된 문장)을 생성함. 통역사가 이해한 의미를 다른 언어로 표현하는 과정과 유사함
- Transformer의 큰 장점 중 하나는 이 인코더와 디코더의 수를 자유롭게 조절할 수 있다는 점. 이를 통해 태스크에 따라 모델의 크기와 복잡도를 조정할 수 있음. 이러한 유연성은 BERT(인코더만 사용)나 GPT(디코더만 사용) 같은 다양한 변형 모델의 탄생과 진화로 이어짐
- 예를 들어 BERT는 주로 텍스트 이해 작업(예: 문장 분류, 질문 답변)에 사용되며, GPT는 주로 텍스트 생성 작업(예: 문장 완성, 창작 글쓰기)에 사용됨. 이처럼 Transformer의 구조적 유연성은 다양한 언어 처리 작업에 최적화된 모델을 만들 수 있게 해주어, AI의 활용 범위를 크게 확장시킴

전이 학습(Transfer Learning)의 용이성

- 대규모 데이터로 사전 학습된 모델을 다양한 하위 태스크에 쉽게 적용할 수 있음

- 이는 NLP 분야에서 전이 학습의 붐을 일으켰고, 적은 데이터로도 높은 성능을 달성함

해석 가능성

- 셀프 어텐션 가중치를 시각화하여 모델의 결정 과정을 이해할 수 있음
- AI의 세세한 처리 과정을 확인하거나 이해할 수 없는 소위 '블랙박스'의 문제를 일부 해소하고, 모델의 신뢰성을 높이는 데 기여함

이러한 특징들 덕분에 법률 문서 분석에서 Transformer는 긴 계약서 전체를 한 번에 처리하고, 멀리 떨어진 조항들 사이의 관계도 파악할 수 있음. 또한, 의료 분야에서는 환자의 긴 병력을 분석하여 현재 증상과 과거 병력 사이의 연관성을 찾아낼 수도 있음

트랜스포머 등장 이후의 변화

대규모 언어 모델의 등장

- GPT, BERT, T5 등 Transformer 기반의 대규모 언어 모델들이 등장
- 이들 모델은 다양한 NLP 태스크에서 인간 수준 이상의 성능을 달성함

전이 학습의 보편화

- 사전 학습된 대규모 모델을 다양한 하위 태스크에 적용하는 방식이 일반화됨
- 이는 적은 양의 데이터로도 높은 성능을 달성할 수 있게 해, AI 기술의 접근성을 크게 개선

멀티모달 AI의 발전

- Transformer의 개념을 이미지, 비디오, 음성 등 다양한 데이터 유형에 적용하기 시작함
- DALL-E, Stable Diffusion 같은 텍스트-이미지 생성 모델이나, WhisperAI 같은 음성 인식 모델 등이 등장

AI의 창의성 향상

- GPT 같은 모델들은 글쓰기, 코딩, 창의적 문제 해결 등에서 놀라운 능력을 보여줌
- 이는 AI가 단순 작업 자동화를 넘어 창의적 영역으로 확장될 수 있음을 보여줌

결론

-
- 트랜스포머는 문장의 의미를 이해하고 생성하는 데 탁월한 능력을 가진 AI 모델
 - 마치 우리가 문장을 읽을 때 각 단어의 관계를 파악하고 전체적인 맥락을 이해하는 것처럼, Transformer도 비슷한 방식으로 작동함
 - 스탠퍼드대학교 연구진은 2021년 8월에 발표한 논문에서 트랜스포머를 “파운데이션 모델(foundation model)”로 일컬은 바 있음. 이 모델이 AI의 패러다임 변화를 견인하고 있다고 봤기 때문
 - 불과 5년 전까지도 가장 인기 있는 딥 러닝 모델로 손꼽혔던 합성곱과 순환 신경망(CNN과 RNN)을 이제는 트랜스포머가 대체하고 있음
 - 실제로 지난 2년간 아카이브(arXiv)에 게재된 AI 관련 논문의 70%에 트랜스포머가 등장함
 - 2017년 전기전자학회(IEEE) 논문에 패턴 인식 분야의 최고 인기 모델로 RNN과 CNN이 보고됐다는 사실을 감안하면 이는 아주 전위적인 변화임
 - 트랜스포머는 요소들 사이의 패턴을 수학적으로 찾아내기 때문에 데이터 세트의 라벨링 과정이 필요 없음
 - 트랜스포머가 사용하는 연산은 병렬 프로세싱에 적합하기 때문에 모델의 실행 속도 또한 빨라짐

참고자료

- <https://wikidocs.net/31379>